PSAT: The Postural Sway Assessment Tool

Developers Manual

Oscar T Giles
o.t.giles@leeds.ac.uk

Raymond J Holt
r.j.holt@leeds.ac.uk

March 2017

Contents

0.1 Overview

The Postural Sway Assessment Tool (PSAT) is an optical motion tracking system for carrying out postural sway assessments. PSAT uses two infra-red (IR) cameras to track head movements and is largely based on the postural sway assessment system developed by Flatters et al., [**?**]. Two IR-LED's are mounted to a pair of glasses which are worn by the participant. When the participant attempts to stand still they make small swaying movements which are captured by the movement of the IR-LED's. These movements are used by PSAT to provide a measure of 'postural sway'. This guide provides an overview of PSAT for developers and advanced users.

PSAT provides a simple measure of postural sway. A participant stands as still as possible while wearing glasses with two infra-red (IR) markers mounted to the frame. PSAT triangulates the position of the IR markers in space, picking up the movement of the markers as the participant sways. This data is then used to provide simple measures of the participant's postural sway.

0.2 General Architecture

PSAT uses two Raspberry Pi 3 Model B's which each control a PiNoIR cameras (see figure 1). The RPi's communicate with each other over an ethernet cable. The *Master* RPi is connected to a touch screen display and provides a graphical user interface (GUI) for users to control PSAT. The *slave* RPi simply controls the right PiNoIR camera as instructed to by the

Master RPi.

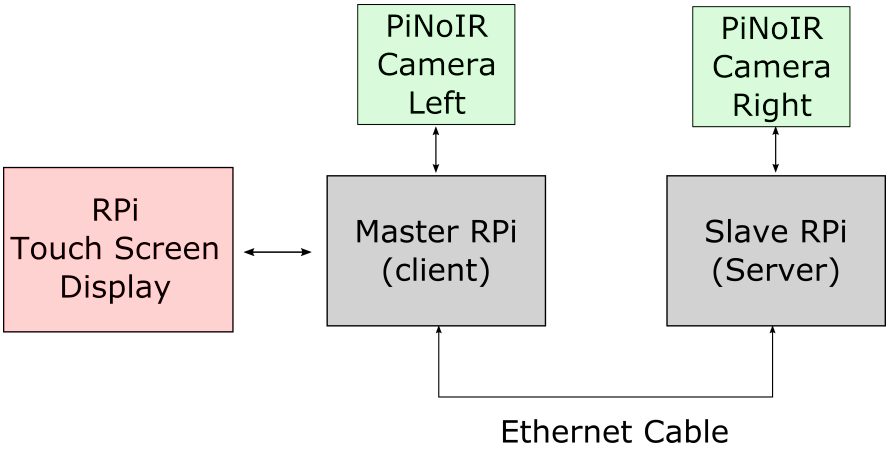

Figure 1: Overview of PSAT hardware architecture.

In essence, PSAT works by recording videos of IR-markers synchronously with both cameras. Computer vision algorithms are then used to identify the IR-markers in the videos and triangulate their positions in 3D space. In addition PSAT provides a simple GUI for recording demographic information and provides data analysis tools.

0.3 Hardware Assembly

PSAT comes readily assembled by Resolve Engineering. However, you can set up you own version by connecting two raspberry pi's by an ethernet cable. Each raspberry pi should have a PiNoIR camera attached. The cameras lenses should be covered by a infra-red pass filter (to filter out visible light). The film from a floppy disk drive works for this purpose quite nicely (see figure 2). Connect a raspberry pi touch screen to the "master (client) RPi".

Figure 2: An image of the raspberry PiNoIR cameras housed in a case with a floppy disk film taped over the lens.

0.4 Developer Installation

PSAT's software can be run on either the Raspberry Pi (running on Raspbian) or on Windows (recording functionality is only available on the RPi - but it is often useful to develop on Windows). First we cover how to set up the development environment on the RPi's and then we examine setting it up on Windows.

0.4.1 Setting up the Raspberry Pi System

The simplest way to install PSAT on the RPi's is to image two 30Gb microSD cards with the *Master* and *Slave* PSAT image files respectively. These images are available on request from the authors and contain the Raspbian operating system along with PSAT and all of its dependencies. Once installed PSAT will boot automatically when the system is switched on. The client image (*'client_master_image.img'*) should be imaged onto the raspberry pi connected to the screen, with the camera on the left hand side (when facing the participant). The slave RPi image (*server_slave_image.img'*) should be imaged onto the remaining RPi (not connected to a display).

PSAT should now function. **But remember to calibrate the cameras for each new set up!!!**

0.4.2 Manual Installation

To install PSAT from scratch on the RPi and to install a development copy on windows requires a little more work.

RPi Manual Install

1. Install the latest version of Raspbian onto a MicroSD card from https://www.raspberrypi.org/downloads/raspbian/

2. Configure the RPi (follow instructions at https://www.raspberrypi.org/documentation/). Ensure that the camera functionality is switched on.

3. PSAT uses Python3 which is already installed on the Raspbian. However, we also require a number of dependencies which must be installed.

Many of the dependencies used by PSAT will already be installed with Raspbian. However the following packages may need to be installed:

1. OpenCV 3.0

2. Numpy

3. psutil

4. picamera

5. matplotlib

6. seaborn

7. numpy

8. pykalman

9. natsort

10. Qt4

11. pandas

The easiest way to install these is using 'pip' or 'apt-get', with the exception of OpenCV 3.0. Installing OpenCV is more complex. Follow the installation instructions here (http://www.pyimagesearch.com/2016/04/18/install-guide-raspberry-pi-3-raspbian-jessie-opencv-3/).

Networking the RPIs

The RPi's must be networked together. PSAT assumes that each RPi has a static IP address (client: 192.168.0.2; server: 192.168.0.2). You can set these up by following the instructions here https://www.modmypi.com/blog/how-to-give-your-rasp You will now be able to communicate between the two RPi's using SHH and move files using SCP.

Placing the PSAT files on the RPis

On the master (client) RPi simply clone a copy of the PSAT repository (available on GitHub) onto a USB stick. Then copy this file to the "Documents" folder on the client. For the slave (server) RPi you need to copy the "PSAT_Server" folder from the PSAT folder into the "Documents" folder on the slave RPi.

If you only have a screen attached to the master (client) RPi you can move files using SCP and control the slave RPi's command line using SSH (see https://www.raspberrypi.org/documentation/remote-access/ for details).

Working on the raspberry pi's

Now you have all the files on the raspberry pi's you probably want to make sure they boot correctly. Using SSH run the "posturalCam_master_NETWORK.py" file on the slave RPi. You can do this using SSH - cd into the correct folder and then execute the file with the command "python3 posturalCam_master_NETWORK.py". This will boot the PSAT server program. On the master RPi simply run the "PSAT.py" file. This should launch the GUI and establish a connection to the client RPi.

Make PSAT run on boot

To make the relevant PSAT files (see last section) run on boot you need to edit a couple of configuration files. On the master pi type the following into the command line:

nano /home/pi/.config/lxsession/LXDE-pi/autostart

and then add this line at the end of the file:

@/usr/bin/python3 /home/pi/Documents/PSAT/PSAT.py

PSAT should now boot when the RPi's are powered on - although it is best to boot the slave (server) RPi first.

Setting up PSAT on the RPi

0.4.3 Manual install on Windows

Installation on windows is fairly painless. Simply download the 64-bit python distribution from Anaconda (https://www.continuum.io/downloads) which will install most of the dependencies that we need. Note however that ana-

conda may now come with Qt5 while Qt4 is required to run PSAT. You can download the correct version of Qt using the anaconda package manager. Installing OpenCV needs to be done manually, but is fairly painless when installing pre-built binaries (see http://docs.opencv.org/3.0-beta/doc/py_tutorials/py_setup/py_setup_in_windows/py_setup_in_windows.html#install-opencv-python-in-windows for instructions).

Clone a copy of PSAT from the github repository. PSAT should now run on your computer. Try executing the "PSAT.py" file which should run a GUI.

0.5 PSAT Code Documentation

To develop PSAT or to run tasks such as calibrate the cameras you will need to work with PSAT's source code (see section 0.4.3). The PSAT working directory has a number of files and folders. To launch PSAT simply run the *PSAT.py* file. The key files that PSAT uses to run are shown in figure 3. The blue box shows the two main files, *posturalCam_master_NETWORK.py* and *PSAT.py*. The former is the backend for PSAT and the later controls the GUI along with some data analysis tasks. The files outside the blue box provide specific functionality for the GUI and the backend. We shall look at each of these scripts in turn.

It's worth noting at this stage the PSAT.py is the entry point for the PSAT data analysis program and the PSAT data collection GUI on the raspberry pi's. The file will detect whether it is running on windows (and load the data analysis program) or the RPi (and run the PSAT data collection

GUI).

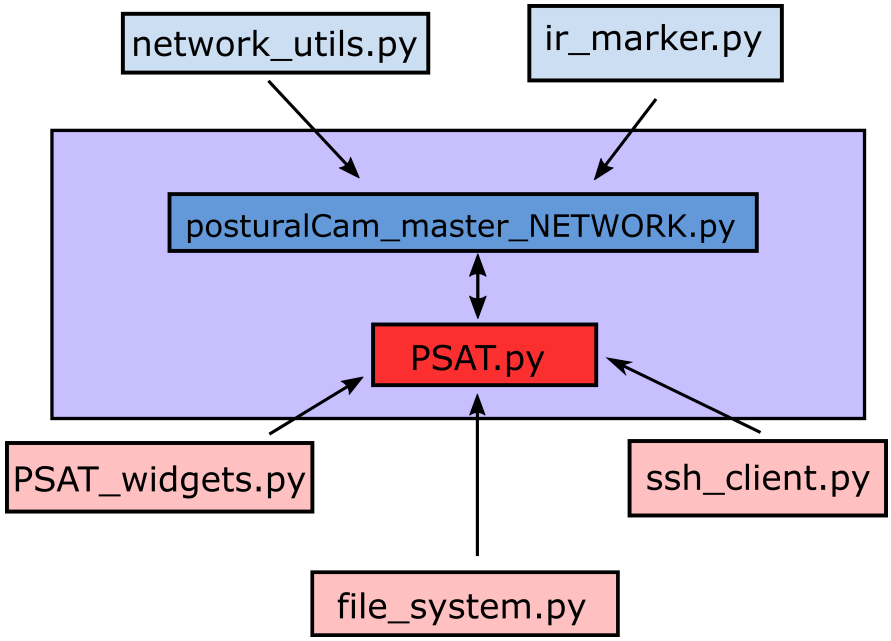

Figure 3: Overview of the PSAT source code files.

0.5.1 The PSAT backend

The file *posturalCam_master_NETWORK.py* provides most of the backend functionality for PSAT. It controls the two IR cameras, communicates between the two RPis and provides data analysis and camera calibration functionality.

This file is the main entry point for PSAT on the *server RPi* which simply executes the *main()* function on boot. On the *master RPi* the backend is used by the *PSAT.py* script as required. On the *server RPi* this file is the entry point to the PSAT server.

Lets look at the file in more details. A number of functions and classes are contained in the file. The key classes are the *posturalCam*, *posturalProc*

and *stereo_process* classes.

posturalCam class

The *posturalCam* class provides methods for controlling the cameras and communicating between the two RPis. It has methods for recoding videos and both the client and server side communication. Communication is done primarily over a TCP server.

A simple program to launch a PSAT server would look like:

```
from postural_cam_master_network import *
myCam = posturalCam()
myCam.TCP_server_start()
```

This will start a TCP server which will wait for a connection from the master RPi. Once a connection has been made it will wait to recieve instructions (see the "TCP_server_start" method for a list of admissible commands).

We can write a simple client side script to send commands to the server:

```
from postural_cam_master_network import *
myCam = posturalCam()
myCam.TCP_client_start()
myCam.TCP_client_start_UDP(t = 10, 'testIR.h264')
```

This creates an instance of the posturalCam class on line 2. Line 3 sends a request to connect to the server. Line 3 then triggers both the client and server RPi's to make a recording for 10 seconds.

We can then ask the server to send over the files of interest:

```
myCam.TCP_client_request_timestamps()
```

```
check_timestamp_error()
myCam.TCP_client_request_video()
myCam.TCP_client_close()
```

The first line asks the server to send over the time stamps, while the second checks the timestamps for any dropped frames. The third line asks the server to send over the video file it took, and the final line closes the connection to the RPi.

This program would leave all the files in the PSAT working directory on the master (client) RPi. The "file_system.py" script (see figure 3) provides functions for saving these files to an appropriate location - and is used by the GUI script (PSAT.py).

posturalProc class

The *posturalProc* class is a class processing a single video file. So we will need to instances to process a video file from the server and the client. The following scrip creates an instance of the posturalProc class, passing the location of the video file and the type of file it is (a recording from the 'client' (master) RPi). The final line of code opens a window which plays the video.

```
from postural_cam_master_network import *
proc = posturalProc(v_fname = 'testIR.h264', kind = 'client')
proc.play()
```

The *posturalProc* class comes with a large number of methods which can be used for different purposes. There are two methods for camera calibration

(see the camera calibration instructions - section 0.6). There are also methods for processing a video to obtain the position if IR LEDs in the image.

stereo_process class

The *stereo_process* class is very similar to the *postural_Proc* class. It contains methods for running the stereo calibration and methods for triangulating IR-LED positions from the two cameras into 3D space. Lets put the *stereo_process* and *posturalProc* classes together in an example script.

Assuming you recoded a video of someone wearing two IR markers (see section 0.5.1), the following code will create two instances of the posturalProc classes in the first two lines.

```
proc = posturalProc(v_fname = 'testIR.h264', kind = 'client')
proc2 = posturalProc(v_fname = 'testIR_server.h264', kind = 'server')

proc_all_markers = ir_marker.markers2numpy(proc.get_ir_markers())
proc2_all_markers = ir_marker.markers2numpy(proc2.get_ir_markers())

stereo =  stereo_process(proc, proc2)
markers3d = stereo.triangulate_all_get_PL(proc_all_markers,
                                          proc2_all_markers)
```

The second to lines process the video files returning a list of dictionaries for each IR data point. The same lines use a function from the "*ir_marker.py*" script to convert the output to a numpy array. The second from last line create a *stereo_process* instance, which takes both the posturalProc instances

as arguments. Finally the last line calls a method which triangulates all the marker positions and returns them as an $N \times M \times 3$-array, where N is the number of video frames and M is the number of markers in the image.

0.5.2 The PSAT GUI

The file "*PSAT.py*" contains the program for the PSAT GUI and is the entry point file on both the master (client) RPi and for the data analysis program on Windows. It is by far the most complex script which creates a GUI interface to the PSAT backend. The GUI relies on QT4, and can run using either the *pyQT4* or *PySide* python bindings to QT4[1].

The *MainWindow* class provides the vast majority of the GUI's functionality and is called when the script is executed. Follow it's *__init__* function to see how it works. The GUI has three main windows which are created by the *self.create_demographics_group*, *self.create_camera_group* and *self.create_processing_group* methods. Examples of these are shown in figures 4, 5 and 6 respectively. It's easiest to look at these methods to see how each window works. Note that windows make use of custom made Widgets which are imported from the *"PSAT_widgets.py"* script.

[1]Please note that PySide is free to use for commercial purposes, while PyQT4 is not. Despite being essentially functionally identical - with a few caveats.

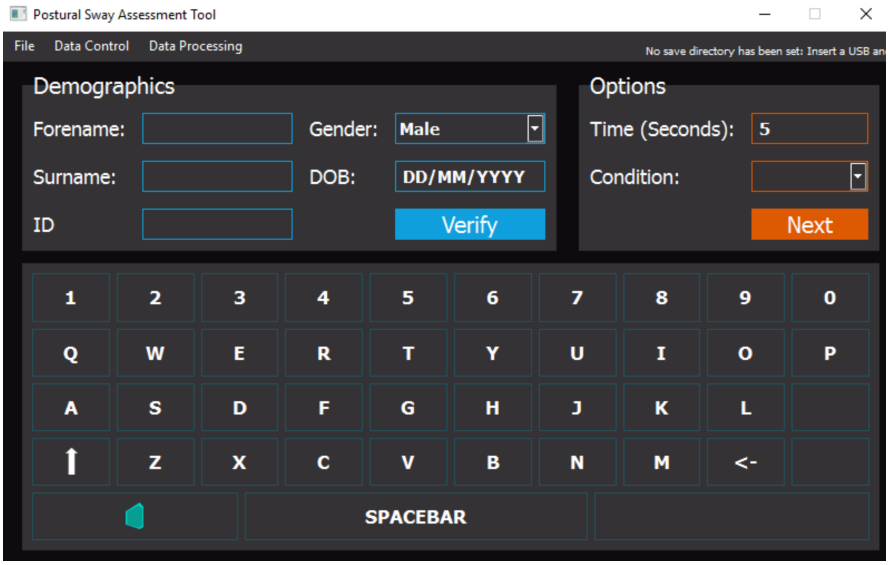

Figure 4: Demographics GUI. See the *self.create_demographics_group* method.

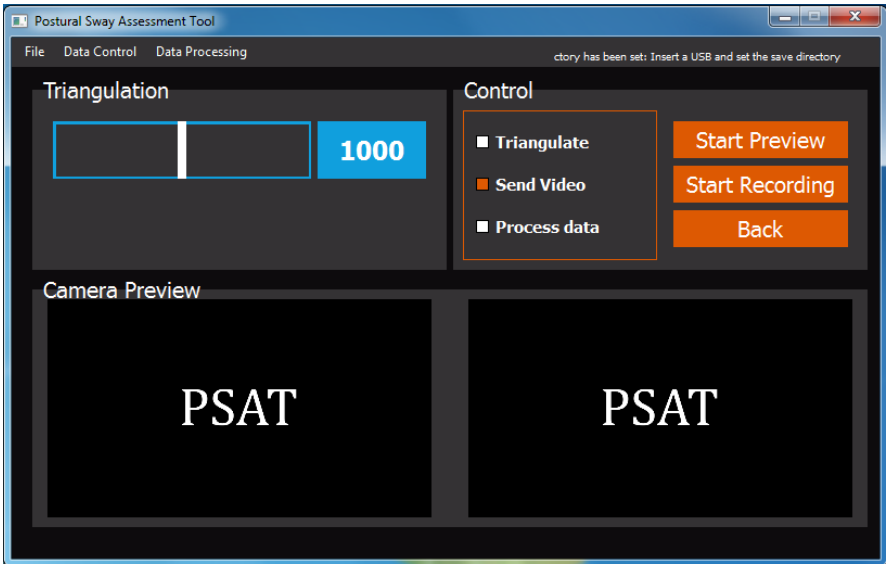

Figure 5: Camera GUI. See the *self.create_camera_group* method.

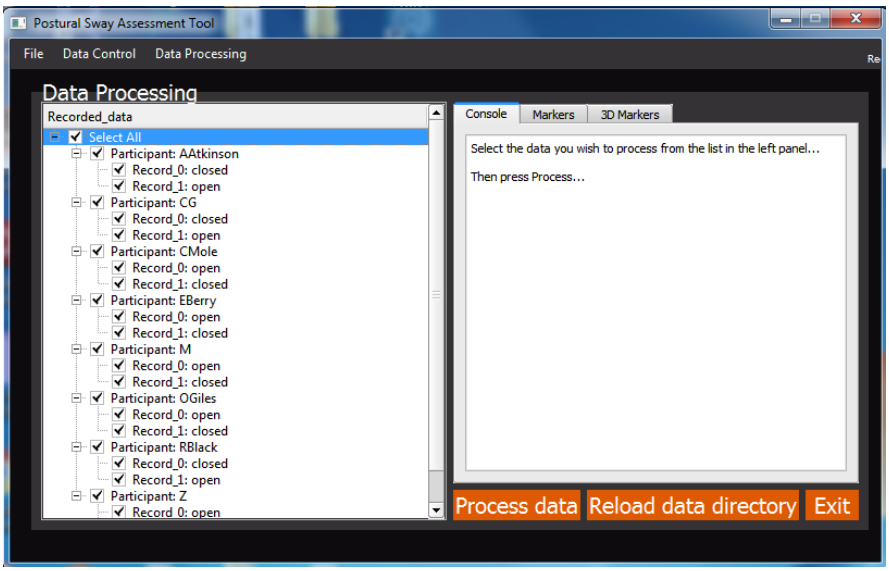

Figure 6: Data Analysis GUI. See the *self.create_processing_group* method.

It is useful to note that the *PSAT.py* will show different behaviour when launched from Windows or on raspberry Pi. On windows it will boot to the data analysis window (figure 6), while on raspberry pi it will boot to the demographics window (figure 4). For a broader overview of what the windows do please see the PSAT user manual.

0.6 Camera Calibration

As documented in the user manual, PSAT requires a USB stick be inserted for data to be recorded. Generally the user will create an empty folder on the USB stick, for example called "*experiment1*". When a recording is made with PSAT is will create a subdirectory in the folder called "*recorded_data*". Each participant then has their own subdirectory, with a further subdirectory for every recording made (called "Record_0", Record_1" etc). Within each record there is a folder called "*calibration*" which contains all the calibration data needed for the recording. The data processing GUI (see figure 6) will look for these files when it processes the data. These calibration files are written to the data directory by copying them from the master (client) RPi's calibration directory.

There are five calibration files altogether (see figure 7). They are stored on the master (client) RPi, under " *./documents/PSAT/calibration*". The three critical ones are called "client_camera_calib_params.pkl", "server_camera_calib_params.pkl" and "stereo_camera_calib_params.pkl". There are also two files called "client_preview_camera_calib and "server_camera_calib_params.pkl".

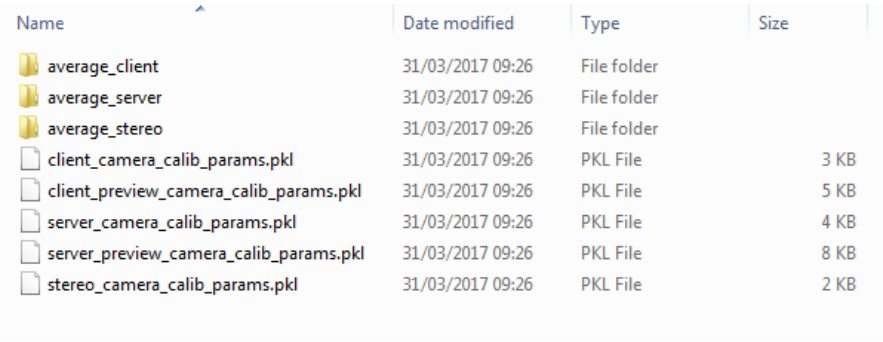

Figure 7: An example calibration folder. There are five calibration files in the folder.

Each PSAT set up needs unique calibration files. When building a new PSAT system or if the camera's on the PSAT system are moved then a camera calibration procedure will need to be performed. The cameras will each need to be calibrated separately and then once again together. The single camera calibrations aim to calibrate the focal length and distortion parameters of the cameras, while the duel calibration seeks to establish the position of the cameras relative to each other. Both are critical for the successful operation of PSAT.

To perform a calibration you will need a) The PSAT system you wish to calibrate and b) A development version of PSAT installed on your computer (see section 0.4.3).

0.6.1 Single Camera Calibration

To calibrate a single camera we need to use a calibration object. This object is a chessboard printed onto an A3 piece of paper and stuck down to a hard backed piece of card (see figure 8).

Figure 8: Calibration objects. A chessboard with 9×6 squares. The squares have a length of 37.5mm. The image shows someone holding up the chessboard for the PSAT cameras. We can see the image in both camera preview screens.

To calibrate the camera first start PSAT and remove the IR-filters from the cameras. Boot PSAT and choose the participant 'ID' as '*Calibration*' (see the user manual for details on how to make recordings with PSAT). You may wish to make several calibration recordings as getting the calibrations right is somewhat of an art form. You can also take multiple recordings and then average over the calibrated results from each calibration (*recommended*). So set the condition name to "c1" and the second recording condition name to 'c2', and so on. For general instructions on how to operate PSAT see the "PSAT User Manual'.

1. When you are ready press record. Hold the calibration object in front of the camera in many different positions and orientations. You want to make sure the calibration object fills a large area of the display - thus it is helpful to do this with a colleague who can view the calibration

object on the preview screen.

2. Repeat this process a number of times. Four or five calibration videos should be adequate.

3. Switch to your computer and navigate to your PSAT development folder (see section 0.4.3). There is a python file called "*camera_calibration.py*". Open it. This file contains a number of functions which we can use to calibrate the cameras (see figure 9).

Figure 9: Screen shot from camera_calibration.py. Select the appropriate function to execute.

4. Insert the USB stick which you saved the calibration videos to (the one you used in PSAT). Find the first calibration record and copy the video files to PSAT development directory (see figure 10).

Figure 10: PSAT development directory with calibration videos copied in

5. Navigate to the subdirectory "calibration", in the PSAT development directory. There are a number of files ending in '*.pkl*' which store the camera calibration details. Delete the files you want to overwrite.

6. Within the "calibration" subdirectory there are three folders called "average_client"... etc. Open these and delete all the *.pkl* files, but not the *.py* files.

7. Go back to the "*camera_calibration_example.py*" script. Scroll to the bottom and you will see three function calls. Uncomment the function

call that describes the calibration you wish to perform. For example, if you are calibrating the camera which will act as the client_camera (i.e. the one connected to the client RPi) uncomment the function "calibrate_client()". Now execute the script!

(a) The script will begin by opening a window with the appropriate video. If you called "calibration_client()" it will load the video client video. If you called "server_calibration()" it will load the server video.

(b) We need to select the images we want to use to do the calibration. Use the 'N' key on your keyboard to skip to the next frame. Use the 'S' key to save that frame for calibration. When you are satisfied that you have enough images press the 'Q' key to end this phase of the calibration. You should aim to have around 20-30 calibration images. Ideally these images should capture the calibration object in different positions (too many images that look the same may result in a poor calibration) - see figure 11.

Figure 11: calibration: Select the images

(c) PSAT will now perform the calibration (see figure 12).

Figure 12: calibration: calibration running

8. A appropriate file will now appear in the "calibration" folder. You can check the calibration by opening the folder 'client' or 'server' which

contains all the images used for the calibration. Follow the next sub-directory to see the images after they have been undistorted. Lines should look straight and there should be no strange distortions.

9. You can now copy the calibration (.pkl) file back into the PSAT folder on the master (client) RPi, which can be found under ". /Documents/P-SAT/Calibration". The calibration is complete.

The exact same instructions can be followed for the server calibration. The stereo calibration is almost identical, except that you will see two screens when selecting images. You should make sure the calibration object is clearly visibly in both screens.

0.7 Building the PSAT data processing executable

To process the PSAT data users will use an executable version of the PSAT GUI. This is created using PyInstaller (http://www.pyinstaller.org/).

1. Install PyInstaller and then navigate to a fresh copy of the PSAT development folder.

2. Navigate to the PSAT development folder in a command window.

3. Call "*pyi-makespec PSAT.py*" in the command window. This will create a file in the PSAT development folder called '*PSAT.spec*'.

4. Open the '*PSAT.spec*' file (see figure 13). Make sure the line that says 'datas=[]' is the same as in figure 13.

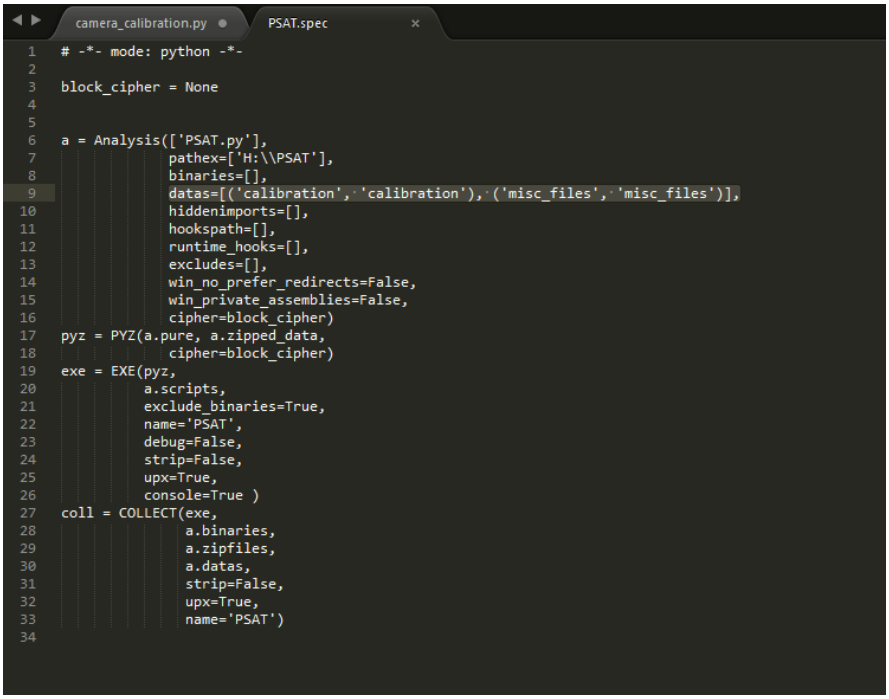

Figure 13: The PSAT Spec file

5. Save the spec file. Go back to the command line and type "*pyinstaller PSAT.spec*.

6. With any luck the code will now compile. When it finished there should be a new folder in the PSAT directory called "dist". Inside is the folder "PSAT" with the distributable folder (containing the executable). Zip up this file and distribute as needed. See the user manual for instructions on how to install.

   Note: Note that the executable will only work on other computers with the same operating system. For example, compile on a 64bit windows 8 computer for use on other 64bit windows computers (it may work with other versions of 64bit windows versions).

0.8 FAQ

Here are a few known issues and problems that you may be asked about by PSAT users...

0.8.1 The data is processing poorly

When the marker data

1. Calibration
2. problems with data processing (set the t_val)
3. Compiling the data analysis script